

第9章 自己写库—构建库函数雏形

本章参考资料：《STM32F4xx 中文参考手册》、《STM32F4xx 英文数据手册》。

虽然我们上面用寄存器编程的方法点亮了 LED，乍看一下好像代码也很简单，但是我们别侥幸以后就可以一直用寄存器开发。在用寄存器点亮 LED 的时候，我们会发现 STM32 的寄存器都是 32 位的，每次配置的时候都要对照着《STM32F4xx 中文参考手册》中寄存器的说明，然后根据说明对每个控制的寄存器位写入特定参数，因此在配置的时候非常容易出错，而且代码还很不好理解，不便于维护。所以学习 STM32 最好的方法是用官方提供的固件库来开发，然后在固件库的基础上了解底层，学遍所有寄存器。

9.1 什么是 STM32 固件库

以上所说的固件库是指“STM32 标准函数库”，它是由 ST 公司针对 STM32 提供的函数接口，即 API (Application Program Interface)，开发者可调用这些函数接口来配置 STM32 的寄存器，使开发人员得以脱离最底层的寄存器操作，有开发快速，易于阅读，维护成本低等优点。

当我们调用库 API 的时候不需要挖空心思去了解库底层的寄存器操作，就像当年我们刚开始学习 C 语言的时候，用 `printf()` 函数时只是学习它的使用格式，并没有去研究它的源码实现，但需要深入研究的时候，经过千锤百炼的库源码就是最佳学习范例。

实际上，库是架设在寄存器与用户驱动层之间的代码，向下处理与寄存器直接相关的配置，向上为用户提供配置寄存器的接口。库开发方式与直接配置寄存器方式的区别见图 9-1。

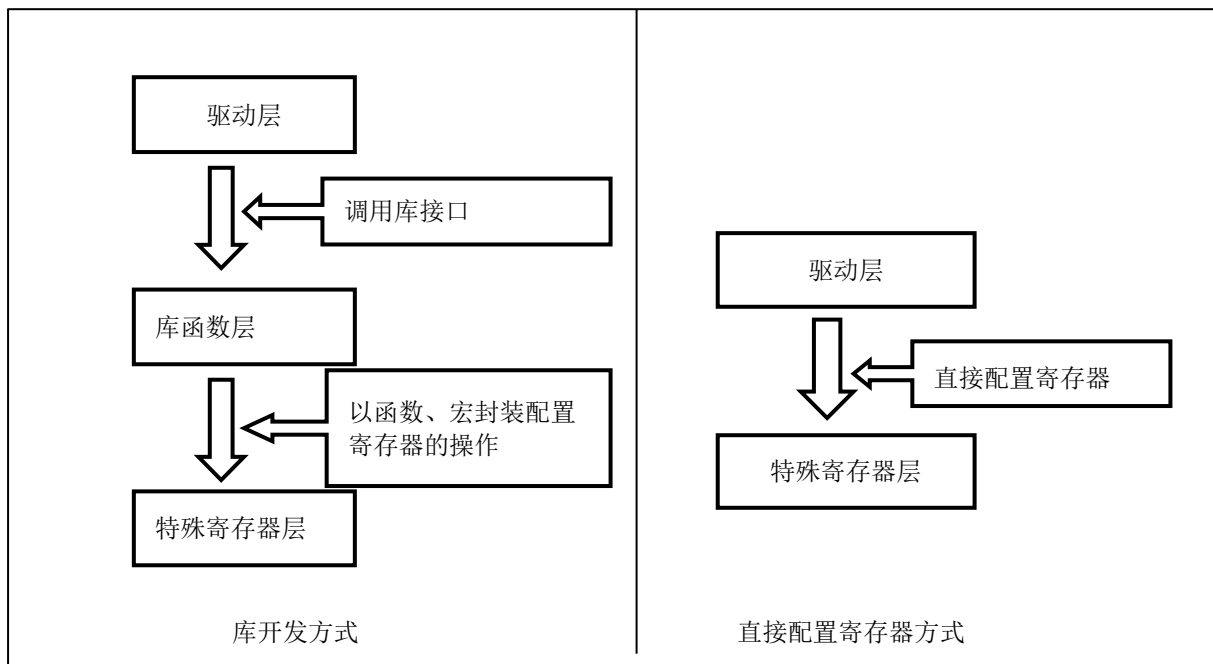


图 9-1 开发方式对比图

9.2 为什么采用库来开发及学习？

在以前 8 位机时代的程序开发中，一般直接配置芯片的寄存器，控制芯片的工作方式，如中断，定时器等。配置的时候，常常要查阅寄存器表，看用到哪些配置位，为了配置某功能，该置 1 还是置 0。这些都是很琐碎的、机械的工作，因为 8 位机的软件相对来说较简单，而且资源很有限，所以可以直接配置寄存器的方式来开发。

对于 STM32，因为外设资源丰富，带来的必然是寄存器的数量和复杂度的增加，这时直接配置寄存器方式的缺陷就突显出来了：

- (1) 开发速度慢
- (2) 程序可读性差
- (3) 维护成本高

这些缺陷直接影响了开发效率，程序维护成本，交流成本。库开发方式则正好弥补了这些缺陷。

而坚持采用直接配置寄存器的方式开发的程序员，会列举以下原因：

- (1) 具体参数更直观
- (2) 程序运行占用资源少

相对于库开发的方式，直接配置寄存器方式生成的代码量的确会少一点，但因为 STM32 有充足的资源，权衡库的优势与不足，绝大部分时候，我们愿意牺牲一点 CPU 资源，选择库开发。一般只有在对代码运行时间要求极苛刻的地方，才用直接配置寄存器的方式代替，如频繁调用的中断服务函数。

对于库开发与直接配置寄存器的方式，就好比编程是用汇编好还是用 C 好一样。在 STM32F1 系列刚推出函数库时引起程序员的激烈争论，但是，随着 ST 库的完善与大家对库的了解，更多的程序员选择了库开发。现在 STM32F1 系列和 STM32F4 系列各有一套自己的函数库，但是它们大部分是兼容的，F1 和 F4 之间的程序移植，只需要小修改即可。

用库来进行开发，市场已有定论，用户群说明了一切，但对于 STM32 的学习仍然有人认为用寄存器好，而且汇编不是还没退出大学教材么？认为这种方法直观，能够了解到是配置了哪些寄存器，怎样配置寄存器。事实上，库函数的底层实现恰恰是直接配置寄存器方式的例子，它代替我们完成了寄存器配置的工作，而想深入了解芯片是如何工作的话，只要直接查看库函数的最底层实现就能理解，相信你会为它严谨、优美的实现方式而陶醉，要想修炼 C 语言，就从 ST 的库开始吧。所以在以后的章节中，使用固件库编程是我们的重点，而且我们通过讲解库函数的实现去高效地学习 STM32 的寄存器，并不至于因为用库学习，就不会用寄存器控制 STM32 芯片。

9.3 实验：构建库函数雏形

虽然库的优点多多，但很多人对库还是很忌惮，因为一开始用库的时候有很多代码，很多文件，不知道如何入手。不知道您是否认同这么一句话：一切的恐惧都来源于认知的空缺。我们对库忌惮那是因为我们不知道什么是库，不知道库是怎么实现的。

接下来，我们在寄存器点亮 LED 的代码上继续完善，把代码一层层封装，实现库的最初的雏形，相信经过这一步的学习后，您对库的运用会游刃有余。这里我们只讲如何实现 GPIO 函数库，其他外设的我们直接参考 ST 标准库学习即可，不必自己写。

下面请打开本章配套例程“自己写库—构建库函数雏形”来阅读理解，该例程是在上一章的基础上修改得来的。

9.3.1 修改寄存器地址封装

上一章中我们在操作寄存器的时候，操作的都是寄存器的绝对地址，如果每个外设寄存器都这样操作，那将非常麻烦。我们考虑到外设寄存器的地址都是基于外设基地址的偏移地址，都是在外设基地址上逐个连续递增的，每个寄存器占 32 个字节，这种方式跟结构体里面的成员类似。所以我们可以定义一种外设结构体，结构体的地址等于外设的基地址，结构体的成员等于寄存器，成员的排列顺序跟寄存器的顺序一样。这样我们操作寄存器的时候就不用每次都找到绝对地址，只要知道外设的基地址就可以操作外设的全部寄存器，即操作结构体的成员即可。

在工程中的“stm32f4xx.h”文件中，我们使用结构体封装 GPIO 及 RCC 外设的寄存器，具体见代码清单 9-1。结构体成员的顺序按照寄存器的偏移地址从低到高排列，成员类型跟寄存器类型一样。

代码清单 9-1 封装寄存器列表

```
1 //寄存器的值常常是芯片外设自动更改的，即使 CPU 没有执行程序，也有可能发生变化
2 //编译器有可能会对没有执行程序的变量进行优化
3
4 //volatile 表示易变的变量，防止编译器优化，
5 #define __IO volatile
6 typedef unsigned int uint32_t;
7 typedef unsigned short uint16_t;
8
9 /* GPIO 寄存器列表 */
10 typedef struct {
11     __IO uint32_t MODER; /*GPIO 模式寄存器 地址偏移: 0x00 */
12     __IO uint32_t OTYPER; /*GPIO 输出类型寄存器 地址偏移: 0x04 */
13     __IO uint32_t OSPEEDR; /*GPIO 输出速度寄存器 地址偏移: 0x08 */
14     __IO uint32_t PUPDR; /*GPIO 上拉/下拉寄存器 地址偏移: 0x0C */
15     __IO uint32_t IDR; /*GPIO 输入数据寄存器 地址偏移: 0x10 */
16     __IO uint32_t ODR; /*GPIO 输出数据寄存器 地址偏移: 0x14 */
17     __IO uint16_t BSRRL; /*GPIO 置位/复位寄存器 低 16 位部分 地址偏移: 0x18 */
18     __IO uint16_t BSRRH; /*GPIO 置位/复位寄存器 高 16 位部分 地址偏移: 0x1A */
19     __IO uint32_t LCKR; /*GPIO 配置锁定寄存器 地址偏移: 0x1C */
20     __IO uint32_t AFR[2]; /*GPIO 复用功能配置寄存器 地址偏移: 0x20-0x24 */
21 } GPIO_TypeDef;
22
23 /*RCC 寄存器列表*/
24 typedef struct {
25     __IO uint32_t CR; /*< RCC 时钟控制寄存器, 地址偏移: 0x00 */
26     __IO uint32_t PLLCFGR; /*< RCC PLL 配置寄存器, 地址偏移: 0x04 */
27     __IO uint32_t CFGR; /*< RCC 时钟配置寄存器, 地址偏移: 0x08 */
28     __IO uint32_t CIR; /*< RCC 时钟中断寄存器, 地址偏移: 0x0C */
29     __IO uint32_t AHB1RSTR; /*< RCC AHB1 外设复位寄存器, 地址偏移: 0x10 */
30     __IO uint32_t AHB2RSTR; /*< RCC AHB2 外设复位寄存器, 地址偏移: 0x14 */
31     __IO uint32_t AHB3RSTR; /*< RCC AHB3 外设复位寄存器, 地址偏移: 0x18 */
32     __IO uint32_t RESERVED0; /*< 保留, 地址偏移: 0x1C */
33     __IO uint32_t APB1RSTR; /*< RCC APB1 外设复位寄存器, 地址偏移: 0x20 */
34     __IO uint32_t APB2RSTR; /*< RCC APB2 外设复位寄存器, 地址偏移: 0x24 */
35     __IO uint32_t RESERVED1[2]; /*< 保留, 地址偏移: 0x28-0x2C*/
36     __IO uint32_t AHB1ENR; /*< RCC AHB1 外设时钟寄存器, 地址偏移: 0x30 */
```

```
37  __IO uint32_t AHB2ENR;          /*!< RCC AHB2 外设时钟寄存器, 地址偏移: 0x34 */
38  __IO uint32_t AHB3ENR;          /*!< RCC AHB3 外设时钟寄存器, 地址偏移: 0x38 */
39  /*RCC 后面还有很多寄存器, 此处省略*/
40 } RCC_TypeDef;
41
42
43 /*定义 GPIOA-H 寄存器结构体指针*/
44 #define GPIOA ((GPIO_TypeDef *) GPIOA_BASE)
45 #define GPIOB ((GPIO_TypeDef *) GPIOB_BASE)
46 #define GPIOC ((GPIO_TypeDef *) GPIOC_BASE)
47 #define GPIOD ((GPIO_TypeDef *) GPIOD_BASE)
48 #define GPIOE ((GPIO_TypeDef *) GPIOE_BASE)
49 #define GPIOF ((GPIO_TypeDef *) GPIOF_BASE)
50 #define GPIOG ((GPIO_TypeDef *) GPIOG_BASE)
51 #define GPIOH ((GPIO_TypeDef *) GPIOH_BASE)
52
53 /*定义 RCC 外设 寄存器结构体指针*/
54 #define RCC ((RCC_TypeDef *) RCC_BASE)
```

这段代码在每个结构体成员前增加了一个“__IO”前缀，它的原型在这段代码的开头，代表了 C 语言中的关键字“volatile”，在 C 语言中该关键字用于修饰易变的变量，要求编译器不要优化。这些结构体内的成员，都代表着寄存器，而寄存器很多时候是由外设或 STM32 芯片状态修改的，也就是说即使 CPU 不执行代码修改这些变量，变量的值也有可能被外设修改、更新，所以每次使用这些变量的时候，我们都要求 CPU 去该变量的地址重新访问。若没有这个关键字修饰，在某些情况下，编译器认为没有代码修改该变量，就直接从 CPU 的某个缓存获取该变量值，这时可以加快执行速度，但该缓存中的是陈旧数据，与我们要求的寄存器最新状态可能会有出入。

9.3.2 定义访问外设的结构体指针

以结构体的形式定义好了外设寄存器后，使用结构体前还需要给结构体的首地址赋值，才能访问到需要的寄存器。为方便操作，我们给每个外设都定义好指向它地址的结构体指针，具体见代码清单 9-2。

代码清单 9-2 指向外设首地址的结构体指针

```
1 /*定义 GPIOA-H 寄存器结构体指针*/
2 #define GPIOA ((GPIO_TypeDef *) GPIOA_BASE)
3 #define GPIOB ((GPIO_TypeDef *) GPIOB_BASE)
4 #define GPIOC ((GPIO_TypeDef *) GPIOC_BASE)
5 #define GPIOD ((GPIO_TypeDef *) GPIOD_BASE)
6 #define GPIOE ((GPIO_TypeDef *) GPIOE_BASE)
7 #define GPIOF ((GPIO_TypeDef *) GPIOF_BASE)
8 #define GPIOG ((GPIO_TypeDef *) GPIOG_BASE)
9 /*定义 RCC 外设 寄存器结构体指针*/
10 #define RCC ((RCC_TypeDef *) RCC_BASE)
```

这些宏通过强制类型转换把外设的基地址转换成 GPIO_TypeDef 类型的指针，从而得到 GPIOA、GPIOB 等直接指向对应外设的指针，通过对结构体指针的操作，即可访问对应外设的寄存器。

利用这些指针访问寄存器，我们把 main 文件里对应的代码修改掉，具体见代码清单 9-3。

代码清单 9-3 使用结构体指针方式控制 LED 灯

```
1 /*
2  * 该章节代码是从 使用寄存器点亮 LED 灯章节代码修改过来
```

```
3  * 属于循序渐进的一章，每一个步骤都对应一部分代码，所以
4  * 该章节代码有好多个部分，不同部分都是使用条件编译来控制
5  * 最后只保留了最后的代码部分。
6  *
7  * 条件编译跟 C 语言的 if else 类似，只有当条件为真的情况下的
8  * 代码才会被编译，所以不要看到有多个 main 函数就以为不正确，
9  * 其实最终编译的只有一个
10 */
11
12 #if 0
13 /* 使用寄存器点亮 LED 灯 章节代码
14 现在已经使用条件编译注释掉 */
15 int main(void)
16 {
17     /*开启 GPIOF 时钟，使用外设时都要先开启它的时钟*/
18     RCC_AHB1ENR |= (1<<5);
19
20     /* LED 端口初始化 */
21
22     /*GPIOF MODER6 清空*/
23     GPIOF_MODER &= ~(0x03<<(2*6));
24     /*PF6 MODER6 = 01b 输出模式*/
25     GPIOF_MODER |= (1<<2*6);
26
27     /*GPIOF OTYPER6 清空*/
28     GPIOF_OTYPER &= ~(1<<1*6);
29     /*PF6 OTYPER6 = 0b 推挽模式*/
30     GPIOF_OTYPER |= (0<<1*6);
31
32     /*GPIOF OSPEEDR6 清空*/
33     GPIOF_OSPEEDR &= ~(0x03<<2*6);
34     /*PF6 OSPEEDR6 = 0b 速率 2MHz*/
35     GPIOF_OSPEEDR |= (0<<2*6);
36
37     /*GPIOF PUPDR6 清空*/
38     GPIOF_PUPDR &= ~(0x03<<2*6);
39     /*PF6 PUPDR6 = 01b 上拉模式*/
40     GPIOF_PUPDR |= (1<<2*6);
41
42     /*PF6 BSRR 寄存器的 BR6 置 1，使引脚输出低电平*/
43     GPIOF_BSRR |= (1<<16<<6);
44
45     /*PF6 BSRR 寄存器的 BS6 置 1，使引脚输出高电平*/
46     //GPIOF_BSRR |= (1<<6);
47
48     while (1);
49 }
50
51 // 函数为空，目的是为了骗过编译器不报错
52 void SystemInit(void)
53 {
54 }
55
56 #elif 1
57
58 /* 自己写库—构建库函数雏形 章节代码
59 现在已经使用条件编译注释掉 */
60 int main(void)
61 {
62     /*开启 GPIOF 时钟，使用外设时都要先开启它的时钟*/
63     RCC->AHB1ENR |= (1<<5);
64
65     /* LED 端口初始化 */
66
```

```
67      /*GPIOF MODER6 清空*/
68      GPIOF->MODER  &= ~( 0x03<< (2*6));
69      /*PF6 MODER6 = 01b 输出模式*/
70      GPIOF->MODER |= (1<<2*6);
71
72      /*GPIOF OTYPER6 清空*/
73      GPIOF->OTYPER &= ~(1<<1*6);
74      /*PF6 OTYPER6 = 0b 推挽模式*/
75      GPIOF->OTYPER |= (0<<1*6);
76
77      /*GPIOF OSPEEDR6 清空*/
78      GPIOF->OSPEEDR &= ~(0x03<<2*6);
79      /*PF6 OSPEEDR6 = 0b 速率 2MHz*/
80      GPIOF->OSPEEDR |= (0<<2*6);
81
82      /*GPIOF PUPDR6 清空*/
83      GPIOF->PUPDR &= ~(0x03<<2*6);
84      /*PF6 PUPDR6 = 01b 上拉模式*/
85      GPIOF->PUPDR |= (1<<2*6);
86
87      /*PF6 BSRR 寄存器的 BR6 置 1，使引脚输出低电平*/
88      GPIOF->BSRRH |= (1<<16<<6);
89
90      /*PF6 BSRR 寄存器的 BS6 置 1，使引脚输出高电平*/
91      //GPIOF->BSRRL |= (1<<6);
92
93      while (1);
94 }
95
96 // 函数为空，目的是为了骗过编译器不报错
97 void SystemInit(void)
98 {
99 }
100
101
102 #endif
```

乍一看，除了把寄存器名字的“-”改成了“->”之外，其他都没有变，但就是这个小变化里面蕴藏这大变化，带“-”的操作是直接操作内存，需要对寄存器的地址一个个进行定义，带“->”的操作是使用外设结构体指针的方式来操作，这为我们继续编写库函数打下了基础。还有一个不同是我们把 BSRR 寄存器分成 BSRRH 和 BSRRL 两段，各 16 位，高 16 位控制复位，低 16 位控制置位，都是写 1 有效。

打好了地基，下面我们就来建高楼。接下来使用函数来封装 GPIO 的基本操作，方便以后应用的时候不需要再查询寄存器，而是直接通过调用这里定义的函数来实现。我们把针对 GPIO 外设操作的函数及其宏定义分别存放在“stm32f4xx_gpio.c”和“stm32f4xx_gpio.h”文件中。

9.3.3 定义位操作函数

在“stm32f4xx_gpio.c”文件定义两个位操作函数，分别用于控制引脚输出高电平和低电平，见代码清单 9-4。

代码清单 9-4 GPIO 置位函数与复位函数的定义

```
1  /**
2   * 函数功能：设置引脚为高电平
3   * 参数说明：GPIOx: 该参数为 GPIO_TypeDef 类型的指针，指向 GPIO 端口的地址
```



```
4  *      GPIO_Pin:选择要设置的 GPIO 端口引脚,可输入宏 GPIO_Pin_0-15,  
5  *      表示 GPIOx 端口的 0-15 号引脚。  
6  */  
7 void GPIO_SetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)  
8 {  
9     /*设置 GPIOx 端口 BSRRL 寄存器的第 GPIO_Pin 位,使其输出高电平*/  
10    /*因为 BSRR 寄存器写 0 不影响,  
11    宏 GPIO_Pin 只是对应位为 1,其它位均为 0,所以可以直接赋值*/  
12  
13    GPIOx->BSRRL = GPIO_Pin;  
14 }  
15  
16 /**  
17  *函数功能: 设置引脚为低电平  
18  *参数说明: GPIOx:该参数为 GPIO_TypeDef 类型的指针,指向 GPIO 端口的地址  
19  *      GPIO_Pin:选择要设置的 GPIO 端口引脚,可输入宏 GPIO_Pin_0-15,  
20  *      表示 GPIOx 端口的 0-15 号引脚。  
21  */  
22 void GPIO_ResetBits(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)  
23 {  
24     /*设置 GPIOx 端口 BSRRH 寄存器的第 GPIO_Pin 位,使其输出低电平*/  
25     /*因为 BSRR 寄存器写 0 不影响,  
26     宏 GPIO_Pin 只是对应位为 1,其它位均为 0,所以可以直接赋值*/  
27  
28     GPIOx->BSRRH = GPIO_Pin;  
29 }
```

这两个函数体内都是只有一个语句,对 GPIOx 的 BSRRL 或 BSRRH 寄存器赋值,从而设置引脚为高电平或低电平。其中 GPIOx 是一个指针变量,通过函数的输入参数我们可以修改它的值,如给它赋予 GPIOA、GPIOB、GPIOH 等结构体指针值,这个函数就可以控制相应的 GPIOA、GPIOB、GPIOH 等端口的输出。

对比我们前面对 BSRR 寄存器的赋值,都是用“|=”操作来防止对其它数据位产生干扰的,为何此函数里的操作却直接用“=”号赋值,这样不怕干扰其它数据位吗?见代码清单 9-5。

代码清单 9-5 赋值方式对比

```
1 /*使用 “|=” 来赋值*/  
2 GPIOF->BSRRH |= (1<<16<<6);  
3 /*直接使用 “=” 号赋值*/  
4 GPIOx->BSRRH = GPIO_Pin;
```

根据 BSRR 寄存器的特性,对它的数据位写“0”,是不会影响输出的,只有对它的数据位写“1”,才会控制引脚输出。对低 16 位写“1”输出高电平,对高 16 位写“1”输出低电平。也就是说,假如我们对 BSRRH(高 16 位)直接用“=”操作赋二进制值“0000 0000 0000 0001 b”,它会控制 GPIO 的引脚 0 输出低电平,赋二进制值“0000 0000 0001 0000 b”,它会控制 GPIO 引脚 4 输出低电平,而其它数据位由于是 0,所以不会受到干扰。同理,对 BSRRL(低 16 位)直接赋值也是如此,数据位为 1 的位输出高电平。代码清单 9-6 中的两种赋值方式,功能相同。

代码清单 9-6 BSRR 寄存器赋值等效代码

```
1 /*使用 “|=” 来赋值*/  
2 GPIOF->BSRRH |= (uint16_t)(1<<6);  
3 /*直接使用“=”来赋值,二进制数(0000 0000 1000 0000)*/
```

```
4 GPIOF->BSRRH = (uint16_t)(1<<6);
```

这两行代码功能等效，都把 BSRRH 的 bit6 设置为 1，控制引脚 6 输出低电平，且其它引脚状态不变。但第二个语句操作效率是比较高的，因为“|=”号包含了读写操作，而“=”号只需要一个写操作。因此在定义位操作函数中我们使用后者。

利用这两个位操作函数，就可以方便地操作 GPIO 的引脚电平了，控制各种端口引脚的范例见代码清单 9-7。

代码清单 9-7 位操作函数使用范例

```
1
2 /*控制 GPIOH 的引脚 10 输出高电平*/
3 GPIO_SetBits(GPIOH, (uint16_t)(1<<10));
4 /*控制 GPIOH 的引脚 10 输出低电平*/
5 GPIO_ResetBits(GPIOH, (uint16_t)(1<<10));
6
7 /*控制 GPIOH 的引脚 10、引脚 11 输出高电平，使用“|”同时控制多个引脚*/
8 GPIO_SetBits(GPIOH, (uint16_t)(1<<10) | (uint16_t)(1<<11));
9 /*控制 GPIOH 的引脚 10、引脚 11 输出低电平*/
10 GPIO_ResetBits(GPIOH, (uint16_t)(1<<10) | (uint16_t)(1<<11));
11
12 /*控制 GPIOA 的引脚 8 输出高电平*/
13 GPIO_SetBits(GPIOA, (uint16_t)(1<<8));
14 /*控制 GPIOB 的引脚 9 输出低电平*/
15 GPIO_ResetBits(GPIOB, (uint16_t)(1<<9));
```

使用以上函数输入参数，设置引脚号时，还是很不方便，且可读性差，为此我们把表示 16 个引脚号的操作数都定义成宏，具体见代码清单 9-8。

代码清单 9-8 选择引脚参数的宏

```
1 /*GPIO 引脚号定义*/
2 #define GPIO_Pin_0          (uint16_t)0x0001  /*!< 选择 Pin0 (1<<0) */
3 #define GPIO_Pin_1          ((uint16_t)0x0002) /*!< 选择 Pin1 (1<<1)*/
4 #define GPIO_Pin_2          ((uint16_t)0x0004) /*!< 选择 Pin2 (1<<2)*/
5 #define GPIO_Pin_3          ((uint16_t)0x0008) /*!< 选择 Pin3 (1<<3)*/
6 #define GPIO_Pin_4          ((uint16_t)0x0010) /*!< 选择 Pin4 */
7 #define GPIO_Pin_5          ((uint16_t)0x0020) /*!< 选择 Pin5 */
8 #define GPIO_Pin_6          ((uint16_t)0x0040) /*!< 选择 Pin6 */
9 #define GPIO_Pin_7          ((uint16_t)0x0080) /*!< 选择 Pin7 */
10 #define GPIO_Pin_8          ((uint16_t)0x0100) /*!< 选择 Pin8 */
11 #define GPIO_Pin_9          ((uint16_t)0x0200) /*!< 选择 Pin9 */
12 #define GPIO_Pin_10         ((uint16_t)0x0400) /*!< 选择 Pin10 */
13 #define GPIO_Pin_11         ((uint16_t)0x0800) /*!< 选择 Pin11 */
14 #define GPIO_Pin_12         ((uint16_t)0x1000) /*!< 选择 Pin12 */
15 #define GPIO_Pin_13         ((uint16_t)0x2000) /*!< 选择 Pin13 */
16 #define GPIO_Pin_14         ((uint16_t)0x4000) /*!< 选择 Pin14 */
17 #define GPIO_Pin_15         ((uint16_t)0x8000) /*!< 选择 Pin15 */
18 #define GPIO_Pin_All        ((uint16_t)0xFFFF) /*!< 选择全部引脚 */
```

这些宏代表的参数是某位置“1”其它位置“0”的数值，其中最后一个

“GPIO_Pin_ALL”是所有数据位都为“1”，所以用它一次可以控制设置整个端口的 0-15 所有引脚。利用这些宏，GPIO 的控制代码可改为代码清单 9-9。

代码清单 9-9 使用位操作函数及宏控制 GPIO

```
1
2 /*控制 GPIOH 的引脚 10 输出高电平*/
3 GPIO_SetBits(GPIOH, GPIO_Pin_10);
```



```
4 /*控制 GPIOH 的引脚 10 输出低电平*/
5 GPIO_ResetBits(GPIOH,GPIO_Pin_10);
6
7 /*控制 GPIOH 的引脚 10、引脚 11 输出高电平,使用“|”,同时控制多个引脚*/
8 GPIO_SetBits(GPIOH,GPIO_Pin_10|GPIO_Pin_11);
9 /*控制 GPIOH 的引脚 10、引脚 11 输出低电平*/
10 GPIO_ResetBits(GPIOH,GPIO_Pin_10|GPIO_Pin_11);
11 /*控制 GPIOH 的所有输出低电平*/
12 GPIO_ResetBits(GPIOH,GPIO_Pin_ALL);
13
14 /*控制 GPIOA 的引脚 8 输出高电平*/
15 GPIO_SetBits(GPIOA,GPIO_Pin_8);
16 /*控制 GPIOB 的引脚 9 输出低电平*/
17 GPIO_ResetBits(GPIOB,GPIO_Pin_9);
```

使用以上代码控制 GPIO, 我们就不需要再看寄存器了, 直接从函数名和输入参数就可以直观看出这个语句要实现什么操作(英文中“Set”表示“置位”, 即高电平, “Reset”表示“复位”, 即低电平)。

9.3.4 定义初始化结构体 GPIO_InitTypeDef

定义位操作函数后, 控制 GPIO 输出电平的代码得到了简化, 但在控制 GPIO 输出电平前还需要初始化 GPIO 引脚的各种模式, 这部分代码涉及的寄存器有很多, 我们希望初始化 GPIO 也能以如此简单的方法去实现。为此, 我们先根据 GPIO 初始化时涉及到的初始化参数以结构体的形式封装起来, 声明一个名为 GPIO_InitTypeDef 的结构体类型, 具体见代码清单 9-10。

代码清单 9-10 定义 GPIO 初始化结构体

```
1 /*
2  * GPIO 初始化结构体类型定义
3  */
4 typedef struct {
5     uint16_t GPIO_Pin;    /* 选择要配置的 GPIO 引脚 */
6     uint8_t  GPIO_Mode;   /* 选择 GPIO 引脚的工作模式 */
7     uint8_t  GPIO_Speed;  /* 选择 GPIO 引脚的速率 */
8     uint8_t  GPIO_OType;  /* 选择 GPIO 引脚输出类型 */
9     uint8_t  GPIO_PuPd;   /* <选择 GPIO 引脚的上/下拉模式 */
10 } GPIO_InitTypeDef;
```

这个结构体中包含了初始化 GPIO 所需要的信息, 包括引脚号、工作模式、输出速率、输出类型以及上/下拉模式。设计这个结构体的思路是: 初始化 GPIO 前, 先定义一个这样的结构体变量, 根据需要配置 GPIO 的模式, 对这个结构体的各个成员进行赋值, 然后把这个变量作为“GPIO 初始化函数”的输入参数, 该函数能根据这个变量值中的内容去配置寄存器, 从而实现 GPIO 的初始化。

9.3.5 定义引脚模式的枚举类型

代码清单 9-10 定义的结构体很直接, 美中不足的是在对结构体中各个成员赋值时还需要看具体哪个模式对应哪个数值, 如 GPIO_Mode 成员的“输入/输出/复用/模拟”模式对应二进制值“00、01、10、11”, 我们不希望每次用到都要去查找这些索引值, 所以使用 C 语言中的枚举语法定义这些参数, 具体见代码清单 9-11。

代码清单 9-11 GPIO 配置参数的枚举定义

```
1 /**
2  * GPIO 端口配置模式的枚举定义
3  */
4 typedef enum {
5     GPIO_Mode_IN   = 0x00, /*!< 输入模式 */
6     GPIO_Mode_OUT  = 0x01, /*!< 输出模式 */
7     GPIO_Mode_AF   = 0x02, /*!< 复用模式 */
8     GPIO_Mode_AN   = 0x03  /*!< 模拟模式 */
9 } GPIOMode_TypeDef;
10
11 /**
12  * GPIO 输出类型枚举定义
13  */
14 typedef enum {
15     GPIO_OType_PP = 0x00, /*!< 推挽模式 */
16     GPIO_OType_OD = 0x01  /*!< 开漏模式 */
17 } GPIOType_TypeDef;
18
19 /**
20  * GPIO 输出速率枚举定义
21  */
22 typedef enum {
23     GPIO_Speed_2MHz   = 0x00, /*!< 2MHz */
24     GPIO_Speed_25MHz  = 0x01, /*!< 25MHz */
25     GPIO_Speed_50MHz  = 0x02, /*!< 50MHz */
26     GPIO_Speed_100MHz = 0x03  /*!<100MHz */
27 } GPIOSpeed_TypeDef;
28
29 /**
30  *GPIO 上/下拉配置枚举定义
31  */
32 typedef enum {
33     GPIO_PuPd_NOPULL = 0x00, /*浮空*/
34     GPIO_PuPd_UP      = 0x01, /*上拉*/
35     GPIO_PuPd_DOWN    = 0x02  /*下拉*/
36 } GPIOPuPd_TypeDef;
```

有了这些枚举定义，我们的 GPIO_InitTypeDef 结构体也可以使用枚举类型来限定输入了，具体见代码清单 9-12。

代码清单 9-12 使用枚举类型定义的 GPIO_InitTypeDef 结构体成员

```
1 /**
2  * GPIO 初始化结构体类型定义
3  */
4 typedef struct {
5     uint32_t GPIO_Pin; /*!< 选择要配置的 GPIO 引脚
6                          可输入 GPIO_Pin_ 定义的宏 */
7
8     GPIOMode_TypeDef GPIO_Mode; /*!< 选择 GPIO 引脚的工作模式
9                                   可输入 GPIOMode_TypeDef 定义的枚举值*/
10
11     GPIOSpeed_TypeDef GPIO_Speed; /*!< 选择 GPIO 引脚的速率
12                                     可输入 GPIOSpeed_TypeDef 定义的枚举值 */
13
14     GPIOType_TypeDef GPIO_OType; /*!< 选择 GPIO 引脚输出类型
15                                     可输入 GPIOType_TypeDef 定义的枚举值*/
16
17     GPIOPuPd_TypeDef GPIO_PuPd; /*!<选择 GPIO 引脚的上/下拉模式
18                                   可输入 GPIOPuPd_TypeDef 定义的枚举值*/
19 } GPIO_InitTypeDef;
```

如果不使用枚举类型，仍使用“uint8_t”类型来定义结构体成员，那么成员值的范围就是 0-255 了，而实际上这些成员都只能输入几个数值。所以使用枚举类型可以对结构体成员起到限定输入的作用，只能输入相应已定义的枚举值。

利用这些枚举定义，给 GPIO_InitTypeDef 结构体类型赋值配置就非常直观了，具体见代码清单 9-13。

代码清单 9-13 给 GPIO_InitTypeDef 初始化结构体赋值范例

```
1 GPIO_InitTypeDef InitStruct;
2
3 /* LED 端口初始化 */
4 /*选择要控制的 GPIO 引脚*/
5 InitStruct.GPIO_Pin = GPIO_Pin_10;
6 /*设置引脚模式为输出模式*/
7 InitStruct.GPIO_Mode = GPIO_Mode_OUT;
8 /*设置引脚的输出类型为推挽输出*/
9 InitStruct.GPIO_OType = GPIO_OType_PP;
10 /*设置引脚为上拉模式*/
11 InitStruct.GPIO_PuPd = GPIO_PuPd_UP;
12 /*设置引脚速率为 2MHz */
13 InitStruct.GPIO_Speed = GPIO_Speed_2MHz;
```

9.3.6 定义 GPIO 初始化函数

接着前面的思路，对初始化结构体赋值后，把它输入到 GPIO 初始化函数，由它来实现寄存器配置。我们的 GPIO 初始化函数实现具体见代码清单 9-14，

代码清单 9-14 GPIO 初始化函数

```
1
2 /**
3  *函数功能：初始化引脚模式
4  *参数说明：GPIOx，该参数为 GPIO_TypeDef 类型的指针，指向 GPIO 端口的地址
5  *          GPIO_InitTypeDef:GPIO_InitTypeDef 结构体指针，指向初始化变量
6  */
7 void GPIO_Init(GPIO_TypeDef* GPIOx, GPIO_InitTypeDef* GPIO_InitStruct)
8 {
9     uint32_t pinpos = 0x00, pos = 0x00 , currentpin = 0x00;
10
11     /*-- GPIO Mode Configuration --*/
12     for (pinpos = 0x00; pinpos < 16; pinpos++) {
13         /*以下运算是为了通过 GPIO_InitStruct->GPIO_Pin 算出引脚号 0-15*/
14
15         /*经过运算后 pos 的 pinpos 位为 1，其余为 0，与 GPIO_Pin_x 宏对应。
16         pinpos 变量每次循环加 1，*/
17         pos = ((uint32_t)0x01) << pinpos;
18
19         /* pos 与 GPIO_InitStruct->GPIO_Pin 做 & 运算，
20         若运算结果 currentpin == pos，
21         则表示 GPIO_InitStruct->GPIO_Pin 的 pinpos 位也为 1，
22         从而可知 pinpos 就是 GPIO_InitStruct->GPIO_Pin 对应的引脚号：0-15*/
23         currentpin = (GPIO_InitStruct->GPIO_Pin) & pos;
24
25         /*currentpin == pos 时执行初始化*/
26         if (currentpin == pos) {
27             /*GPIOx 端口，MODER 寄存器的 GPIO_InitStruct->GPIO_Pin 对应的引脚，
28             MODER 位清空*/
29             GPIOx->MODER &= ~(3 << (2 *pinpos));
30
```

```
31      /*GPIOx 端口, MODER 寄存器的 GPIO_Pin 引脚,
32      MODER 位设置"输入/输出/复用输出/模拟"模式*/
33      GPIOx->MODER |= (((uint32_t)GPIO_InitStruct->GPIO_Mode) << (2 *pinpos));
34
35      /*GPIOx 端口, PUPDR 寄存器的 GPIO_Pin 引脚,
36      PUPDR 位清空*/
37      GPIOx->PUPDR &= ~(3 << ((2 *pinpos)));
38
39      /*GPIOx 端口, PUPDR 寄存器的 GPIO_Pin 引脚,
40      PUPDR 位设置"上/下拉"模式*/
41      GPIOx->PUPDR |= (((uint32_t)GPIO_InitStruct->GPIO_PuPd) << (2 *pinpos));
42
43      /*若模式为"输出/复用输出"模式, 则设置速度与输出类型*/
44      if ((GPIO_InitStruct->GPIO_Mode == GPIO_Mode_OUT) ||
45          (GPIO_InitStruct->GPIO_Mode == GPIO_Mode_AF)) {
46          /*GPIOx 端口, OSPEEDR 寄存器的 GPIO_Pin 引脚,
47          OSPEEDR 位清空*/
48          GPIOx->OSPEEDR &= ~(3 << (2 *pinpos));
49          /*GPIOx 端口, OSPEEDR 寄存器的 GPIO_Pin 引脚,
50          OSPEEDR 位设置输出速度*/
51          GPIOx->OSPEEDR |= ((uint32_t)(GPIO_InitStruct->GPIO_Speed)<<(2 *pinpos));
52
53          /*GPIOx 端口, OTyPER 寄存器的 GPIO_Pin 引脚,
54          OTyPER 位清空*/
55          GPIOx->OTyPER &= ~(1 << (pinpos)) ;
56          /*GPIOx 端口, OTyPER 寄存器的 GPIO_Pin 引脚,
57          OTyPER 位设置"推挽/开漏"输出类型*/
58          GPIOx->OTyPER |= (uint16_t)(( GPIO_InitStruct->GPIO_OType)<< (pinpos));
59      }
60  }
61  }
62 }
```

这个函数有 GPIOx 和 GPIO_InitStruct 两个输入参数, 分别是 GPIO 外设指针和 GPIO 初始化结构体指针。分别用来指定要初始化的 GPIO 端口及引脚的工作模式。

函数实现主要分两个环节:

- (1) 利用 for 循环, 根据 GPIO_InitStruct 的结构体成员 GPIO_Pin 计算出要初始化的引脚号。这段看起来复杂的运算实际上可以这样理解: 它要通过宏“GPIO_Pin_x”的参数计算出 x 值(宏的参数值是第 x 数据位为 1, 其余为 0, 参考代码清单 9-8), 计算得的引脚号结果存储在 pinpos 变量中。
- (2) 得到引脚号 pinpos 后, 利用初始化结构体各个成员的值, 对相应寄存器进行配置, 这部分与我们前面直接配置寄存器的操作是类似的, 先对引脚号 pinpos 相应的配置位清空, 后根据结构体成员对配置位赋值(GPIO_Mode 成员对应 MODER 寄存器的配置, GPIO_PuPd 成员对应 PUPDR 寄存器的配置等)。区别是这里的寄存器配置值及引脚号都是由变量存储的。

9.3.7 全新面貌, 使用函数点亮 LED 灯

完成以上的准备后, 我们就可以用自己定义的函数来点亮 LED 灯了, 具体见代码清单 9-15。

代码清单 9-15 使用函数点亮 LED 灯

```
1 /* 自己写库—构建库函数雏形 最终代码 */
2
3 void Delay( uint32_t nCount);
4
5 int main(void)
6 {
7     GPIO_InitTypeDef InitStruct;
8
9     /*开启 GPIOF 时钟, 使用外设时都要先开启它的时钟*/
10    RCC->AHB1ENR |= (1<<5);
11
12    /* LED 端口初始化 */
13
14    /*初始化 PF6 引脚*/
15    /*选择要控制的 GPIO 引脚*/
16    InitStruct.GPIO_Pin = GPIO_Pin_6;
17    /*设置引脚模式为输出模式*/
18    InitStruct.GPIO_Mode = GPIO_Mode_OUT;
19    /*设置引脚的输出类型为推挽输出*/
20    InitStruct.GPIO_OType = GPIO_OType_PP;
21    /*设置引脚为上拉模式*/
22    InitStruct.GPIO_PuPd = GPIO_PuPd_UP;
23    /*设置引脚速率为 2MHz */
24    InitStruct.GPIO_Speed = GPIO_Speed_2MHz;
25    /*调用库函数, 使用上面配置的 GPIO_InitStructure 初始化 GPIO*/
26    GPIO_Init(GPIOF, &InitStruct);
27
28    /*使引脚输出低电平, 点亮 LED1*/
29    GPIO_ResetBits(GPIOF, GPIO_Pin_6);
30
31    /*延时一段时间*/
32    Delay(0xFFFFF);
33
34    /*使引脚输出高电平, 关闭 LED1*/
35    GPIO_SetBits(GPIOF, GPIO_Pin_6);
36
37    /*初始化 PF7 引脚*/
38    InitStruct.GPIO_Pin = GPIO_Pin_7;
39    GPIO_Init(GPIOF, &InitStruct);
40
41    /*使引脚输出低电平, 点亮 LED2*/
42    GPIO_ResetBits(GPIOF, GPIO_Pin_7);
43
44    while (1);
45 }
46
47 //简单的延时函数, 让 cpu 执行无意义指令, 消耗时间
48 //具体延时时间难以计算, 以后我们可使用定时器精确延时
49 void Delay( uint32_t nCount)
50 {
51     for (; nCount != 0; nCount--);
52 }
53
54
55 // 函数为空, 目的是为了骗过编译器不报错
56 void SystemInit(void)
57 {
58 }
59 }
```

现在看起来, 使用函数来控制 LED 灯与之前直接控制寄存器已经有了很大的区别: main 函数中先定义了一个初始化结构体变量 InitStruct, 然后对该变量的各个成员按点亮 LED 灯所需要的 GPIO 配置模式进行赋值, 赋值后, 调用 GPIO_Init 函数, 让它根据结构

体成员值对 GPIO 寄存器写入控制参数，完成 GPIO 引脚初始化。控制电平时，直接使用 GPIO_SetBits 和 GPIO_Resetbits 函数控制输出。如若对其它引脚进行不同模式的初始化，只要修改初始化结构体 InitStruct 的成员值，把新的参数值输入到 GPIO_Init 函数再调用即可。

代码中新增的 Delay 函数，主要功能是延时，让我们可以看清楚实验现象(不延时的话指令执行太快，肉眼看不出来)，它的实现原理是让 CPU 执行无意义的指令，消耗时间，在此不要纠结它的延时时间，写一个大概输入参数值，下载到实验板实测，觉得太久了就把参数值改小，短了就改大即可。需要精确延时的时候我们会用 STM32 的定时器外设进行精确延时的。

9.3.8 下载验证

把编译好的程序下载到开发板并复位，可看到板子上的灯先亮红色(LED1)，后亮绿色(LED2)。

9.3.9 总结

什么是 ST 固件库？我们上面写的就是，相对于官方的完整版的固件库，我们写的只是一个雏形，写这个固件库的雏形目的是为了帮助我们从容寄存器编程顺利地过度到到固件库编程的，让我们知道什么是固件库，为以后能够熟练的使用固件库编程打下基础。

我们从寄存器映射开始，把内存跟寄存器建立起一一对应的关系，然后操作寄存器点亮 LED，再把寄存器操作封装成一个个函数。一步一步走来，我们实现了库最简单的雏形，如果我们不断地增加操作外设的函数，并且把所有的外设都写完，一个完整的库就实现了。

本章中的 GPIO 相关库函数及结构体定义，实际上都是从 ST 标准库搬过来的。这样分析它纯粹是为了满足自己的求知欲，学习其编程的方式、思想，这对提高我们的编程水平是很有好处的，顺便感受一下 ST 库设计的严谨性，我认为这样的代码不仅严谨且华丽优美，不知你是否也有这样的感受。

与直接配置寄存器相比，从执行效率上看会有额外的消耗：初始化变量赋值的过程、库函数在被调用的时候要耗费调用时间；在函数内部，对输入参数转换所需要的额外运算也消耗一些时间(如 GPIO 中运算求出引脚号时)。而其它的宏、枚举等解释操作是在编译过程完成的，这部分并不消耗内核的时间。那么函数库的优点呢？是我们可以快速上手 STM32 控制器；配置外设状态时，不需要再纠结要向寄存器写入什么数值；交流方便，查错简单。这就是我们选择库的原因。

现在的处理器的主频是越来越高，我们不需要担心 CPU 耗费那么多时间来干活会不会被累倒，库主要应用是在初始化过程，而初始化过程一般是芯片刚上电或在核心运算之前的执行的，这段时间的等待是 0.02us 还是 0.01us 在很多时候并没有什么区别。相对来说，我们还是担心一下如果都用寄存器操作，每行代码都要查《STM32F4xx 中文参考手册》中的寄存器说明，自己会不会被累倒吧。

在以后开发的工程中，一般不会去分析 ST 的库函数的实现了。因为外设的库函数是很类似的，库外设都包含初始化结构体，以及特定的宏或枚举标识符，这些封装被库函数

这些转化成相应的值，写入到寄存器之中，函数内部的具体实现是十分枯燥和机械的工作。如果你有兴趣，在你掌握了如何使用外设的库函数之后，可以查看一下它的源码实现。

通常我们只需要通过了解每种外设的“初始化结构体”就能够通过它去了解 STM32 的外设功能及控制了。